

Ponteiros

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- A memória do computador é organizada como uma tabela
- Imagine um computador hipotético onde a memória do computador é endereçada em bytes
- Cada linha da tabela possui um endereço, e em cada endereço podemos armazenar um byte
- Toda variável, ao ser criada, é associada a um endereço

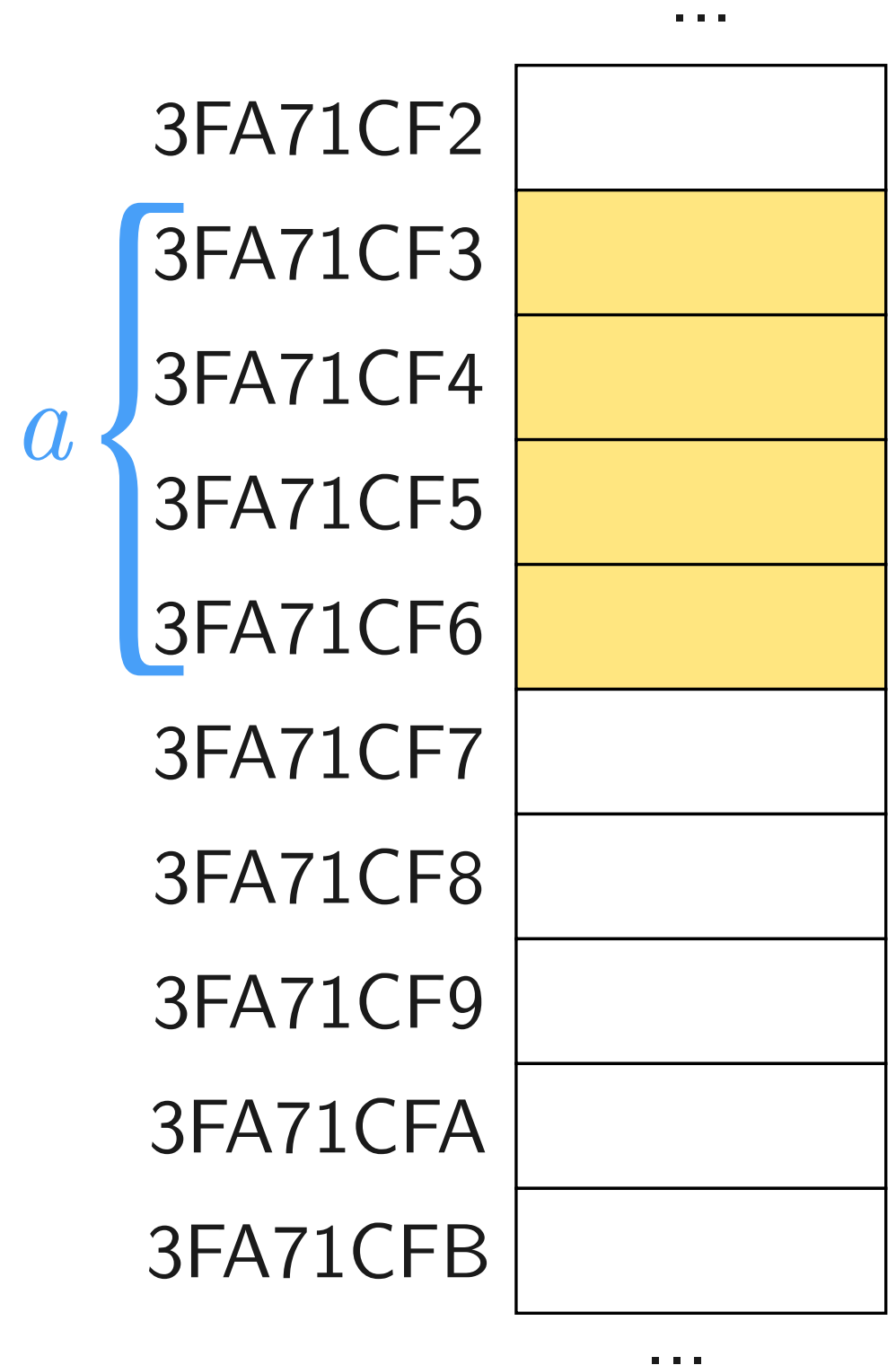
...

3FA71CF2	
3FA71CF3	
3FA71CF4	
3FA71CF5	
3FA71CF6	
3FA71CF7	
3FA71CF8	
3FA71CF9	
3FA71CFA	
3FA71CFB	

...

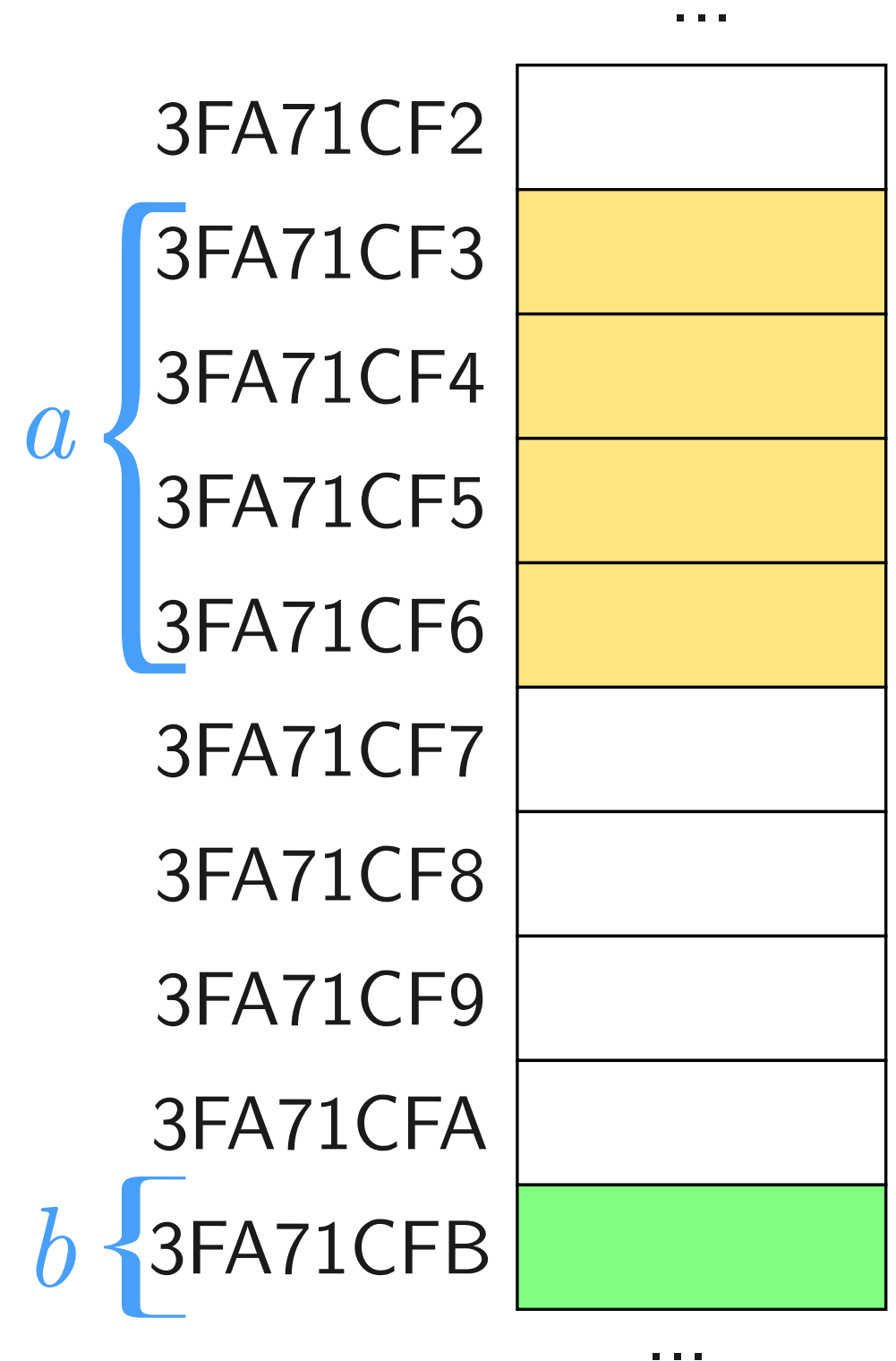
Variáveis na Memória

```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    return 0;  
}
```



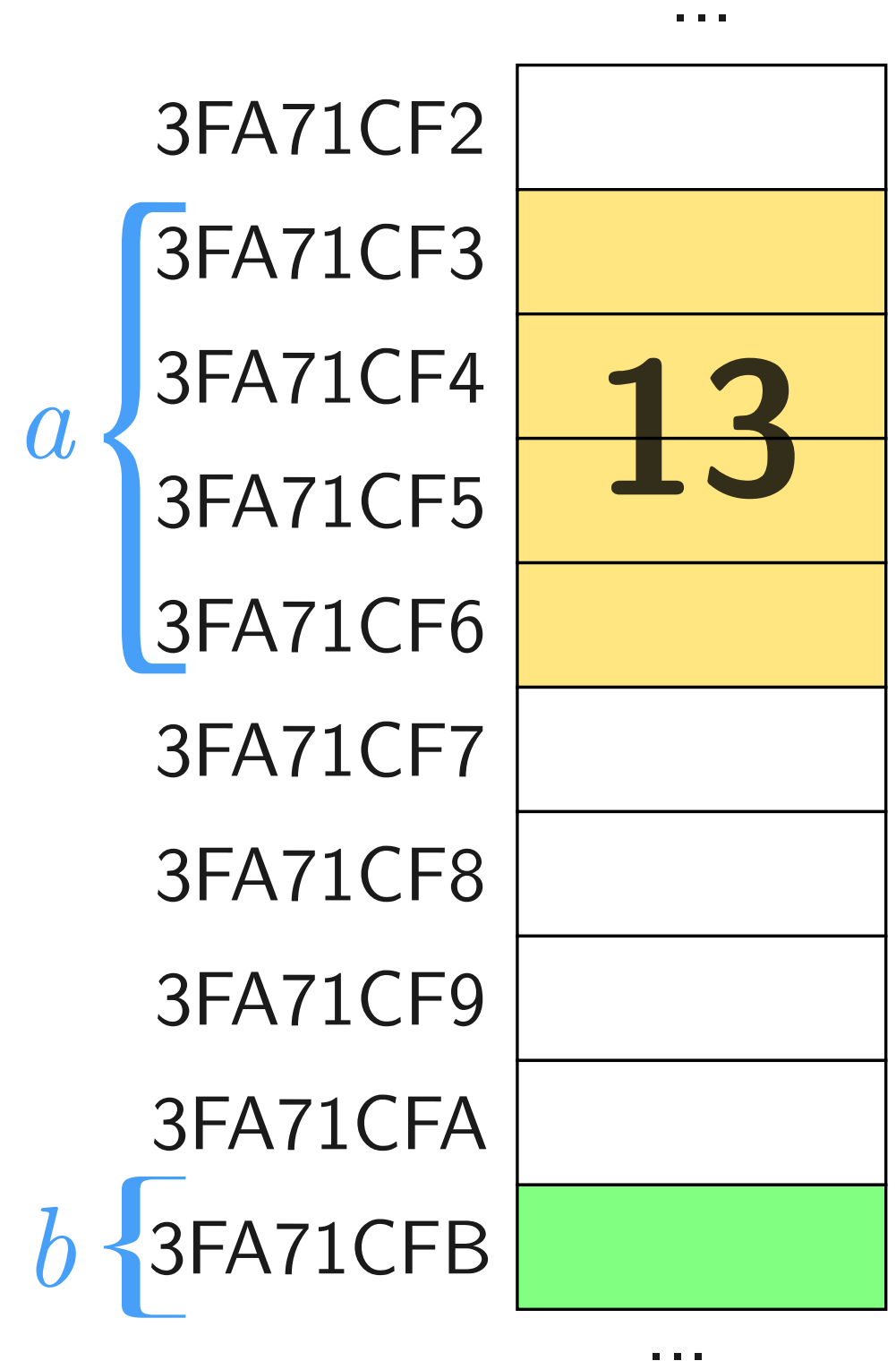
Variáveis na Memória (2)

```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    return 0;  
}
```



Variáveis na Memória (3)

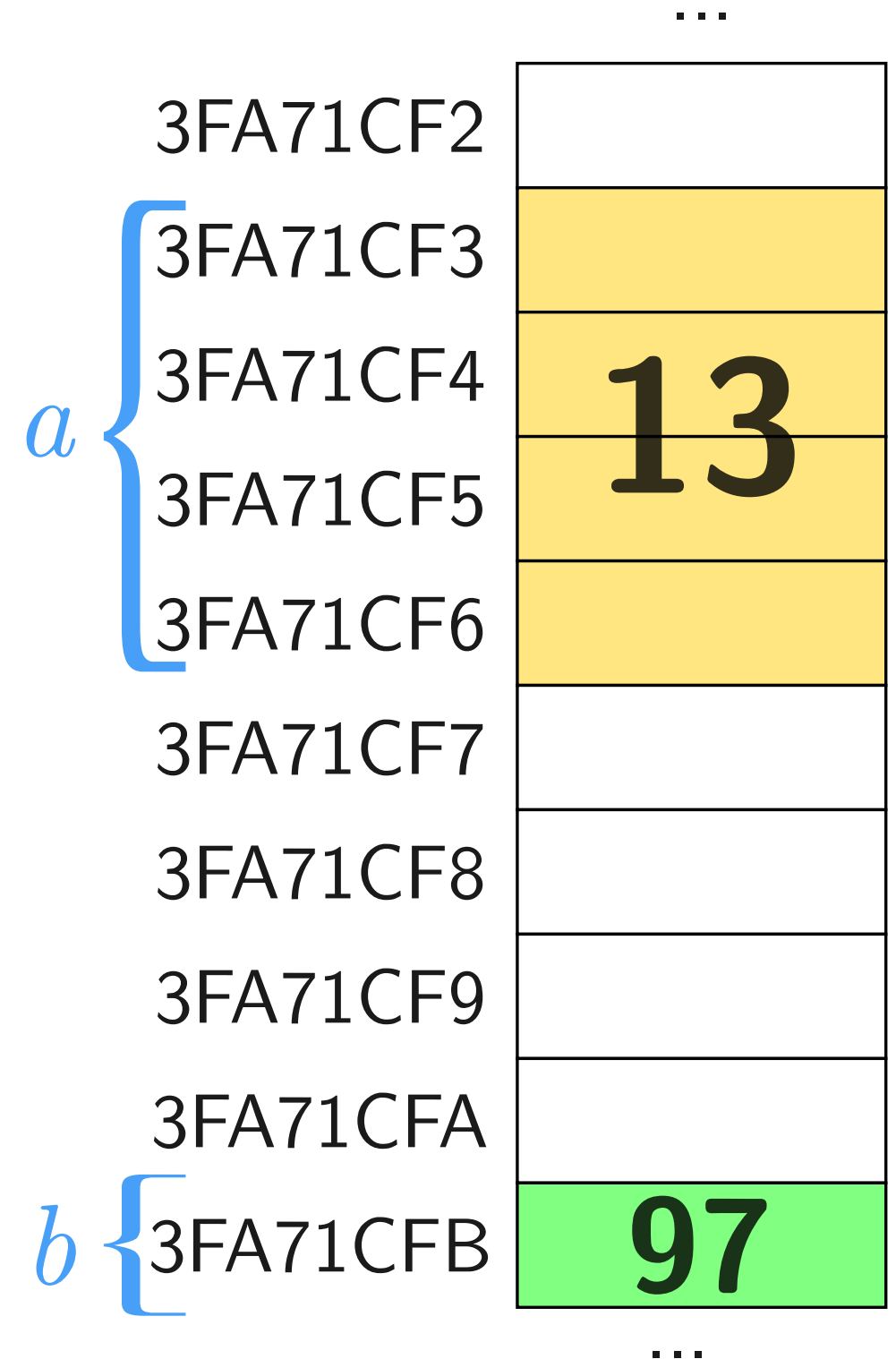
```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    return 0;  
}
```



Variáveis na Memória (4)

```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    return 0;  
}
```

b = 'a';



Espaço Alocado na Memória

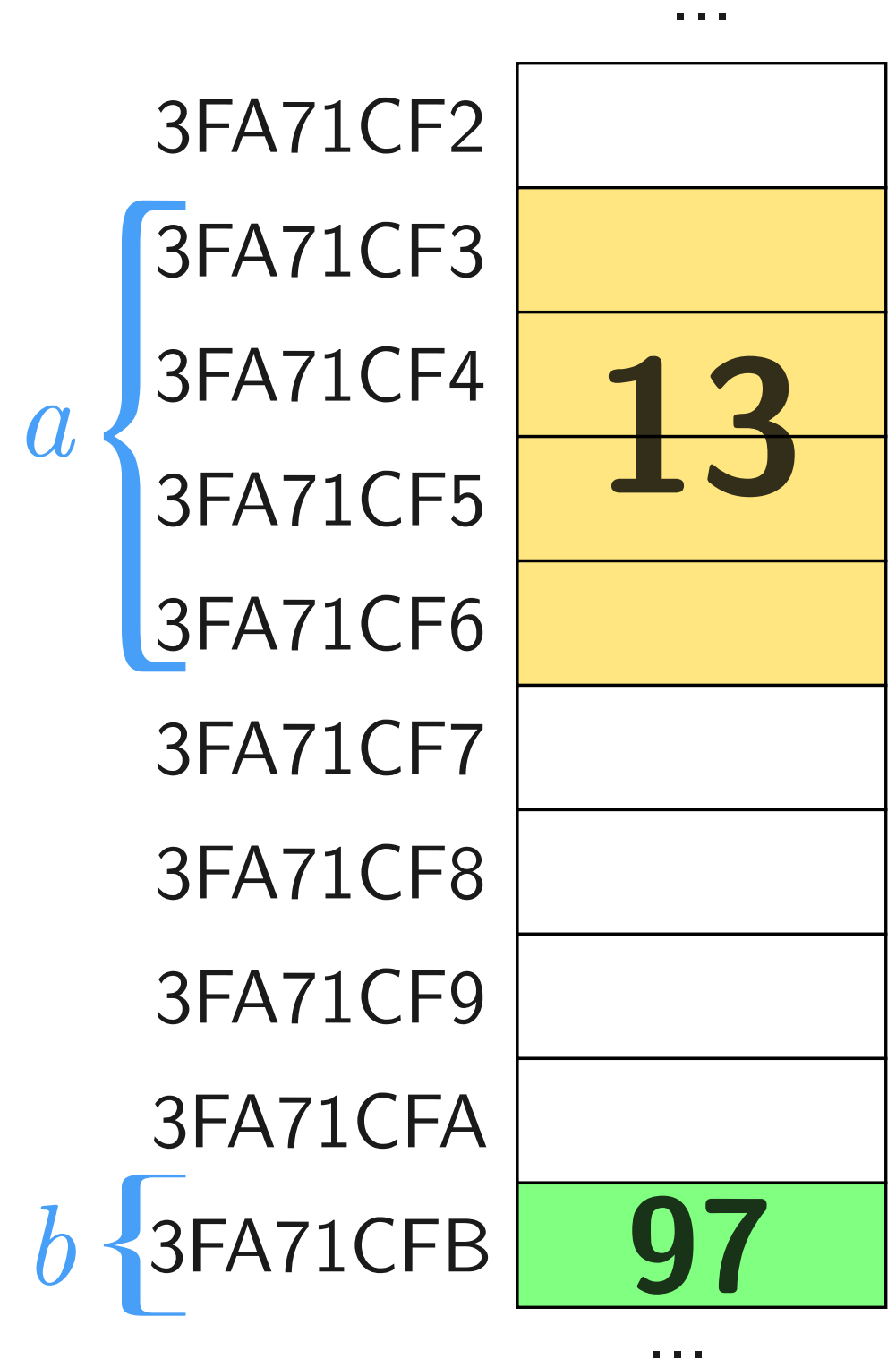
Tipo	Bits	Bytes	Escala
char	8	1	128 a 127
int	32	4	-2.147.483.648 a 2.147.483.647 (ambientes de 32 bits)
short	16	2	-32.765 a 32.767
long	32	4	-2.147.483.648 a 2.147.483.647
unsigned char	8	1	0 a 255
unsigned	32	4	0 a 4.294.967.295 (ambientes de 32 bits)
unsigned long	32	4	0 a 4.294.967.295
unsigned short	16	2	0 a 65.535
float	32	4	$3,4 \times 10^{-38}$ a $3,4 \times 10^{38}$
double	64	8	$1,7 \times 10^{-308}$ a $1,7 \times 10^{308}$
long double	80	10	$3,4 \times 10^{-4932}$ a $3,4 \times 10^{4932}$
void	0	0	nenhum valor

Tipos de variáveis em C.

Conteúdo e Endereço da Variável

```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    printf("%d\n", a);  
  
    return 0;  
}
```

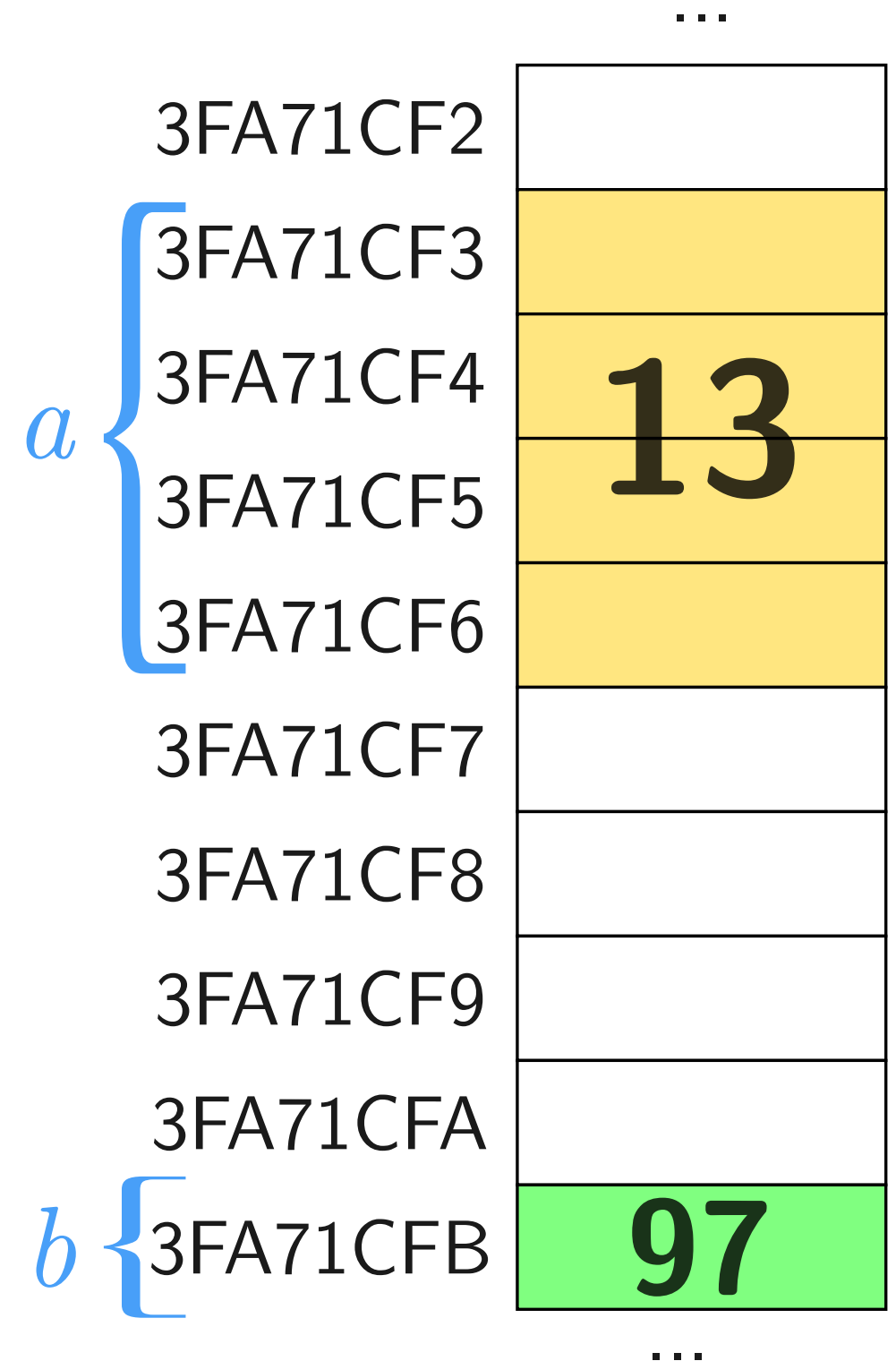
13



Conteúdo e Endereço da Variável (Cont.)

```
int main () {  
    int a;  
    char b;  
  
    a = 13;  
    b = 'a';  
  
    printf("%d\n", a);  
    printf("%p\n", &a);  
  
    return 0;  
}
```

13
3FA71CF3



Ponteiros

- **Ponteiro** é um tipo de variável que armazena um endereço
- Para que serve?
 1. Alocar memória dinamicamente
 2. Simular chamada por referência
Obs.: em C++ existe uma maneira mais fácil de passar por referência
 3. Manipularmos estruturas de dados dinâmicas

Ponteiros: Declaração

- A declaração de um ponteiro é feita com a adição do **símbolo *** (asterisco) na declaração
- **p** é um ponteiro para **int**

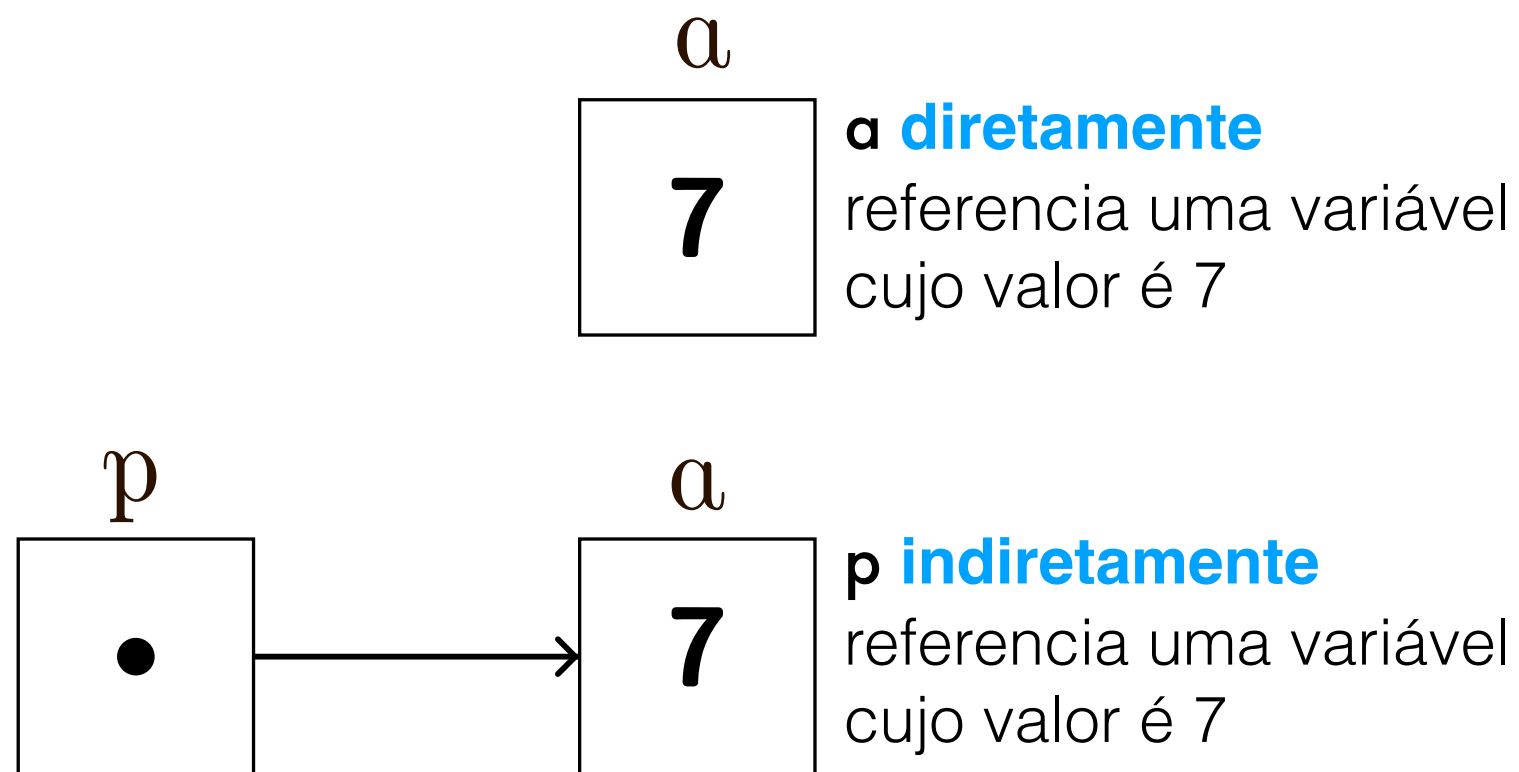
```
int main () {  
    int *p, a;  
  
    return 0;  
}
```

Erro Comum de Programação 1

A notação asterisco (*) utilizada para declarar ponteiros não é distribuída para todas variáveis da declaração. Cada ponteiro deve ser declarado com um * prefixado no nome; e.g., se você deseja declarar **xPtr** e **yPtr** como ponteiros do tipo inteiro, utilize:

```
int main () {  
    int *xPtr, *yPtr;  
  
    return 0;  
}
```

Referência Direta e Indireta

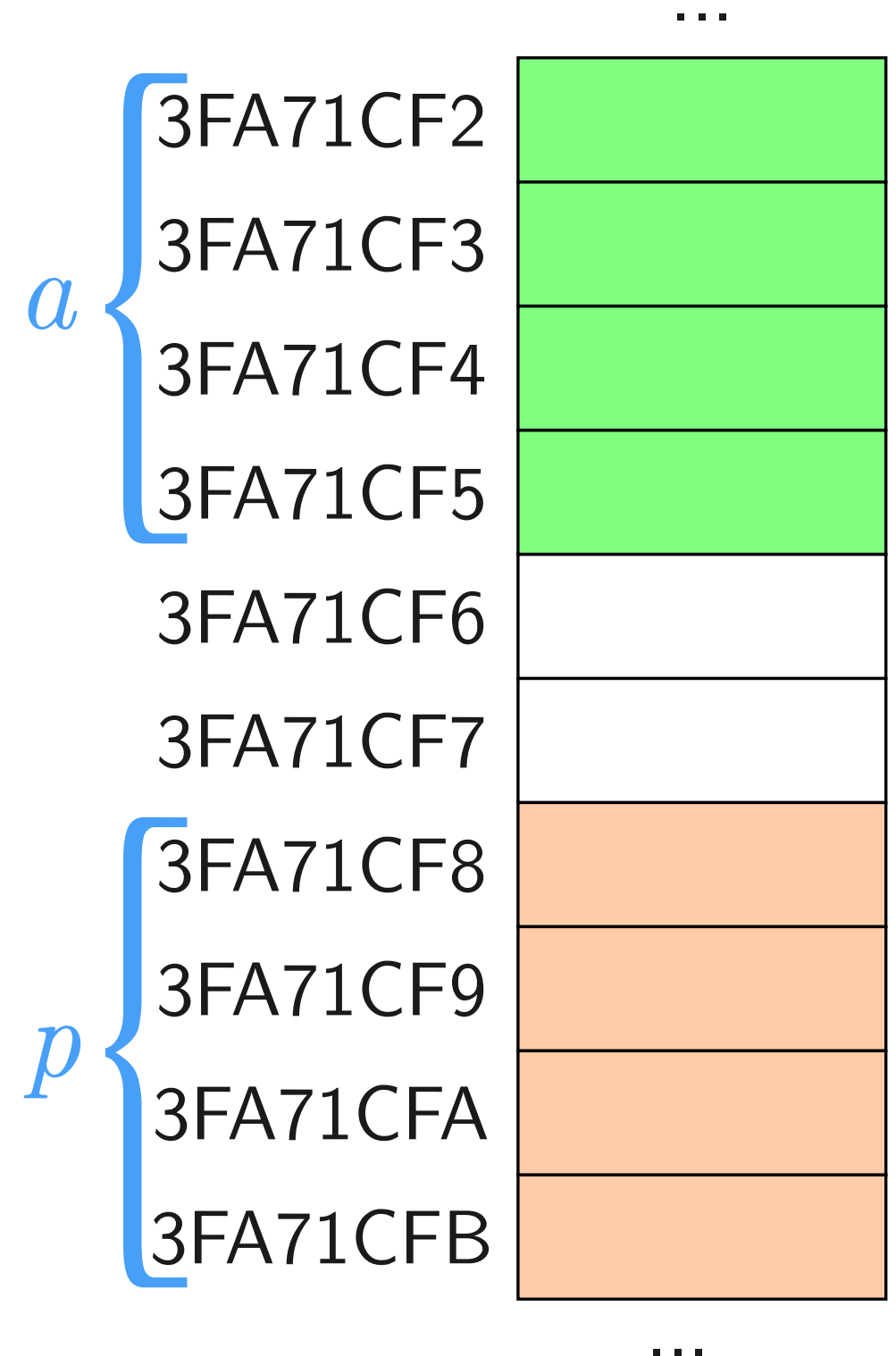


Ponteiros: Inicialização

- Ponteiros devem ser inicializados:
 1. Quando são declarados ou
 2. Em uma expressão de atribuição
- Um ponteiro pode ser inicializado com **0**, **NULL** ou um endereço
 - Um ponteiro com o valor **0** ou **NULL** aponta para nada
 - Para obtermos endereços de variáveis utilizamos o operador **&**

Atribuição Endereço a Ponteiro

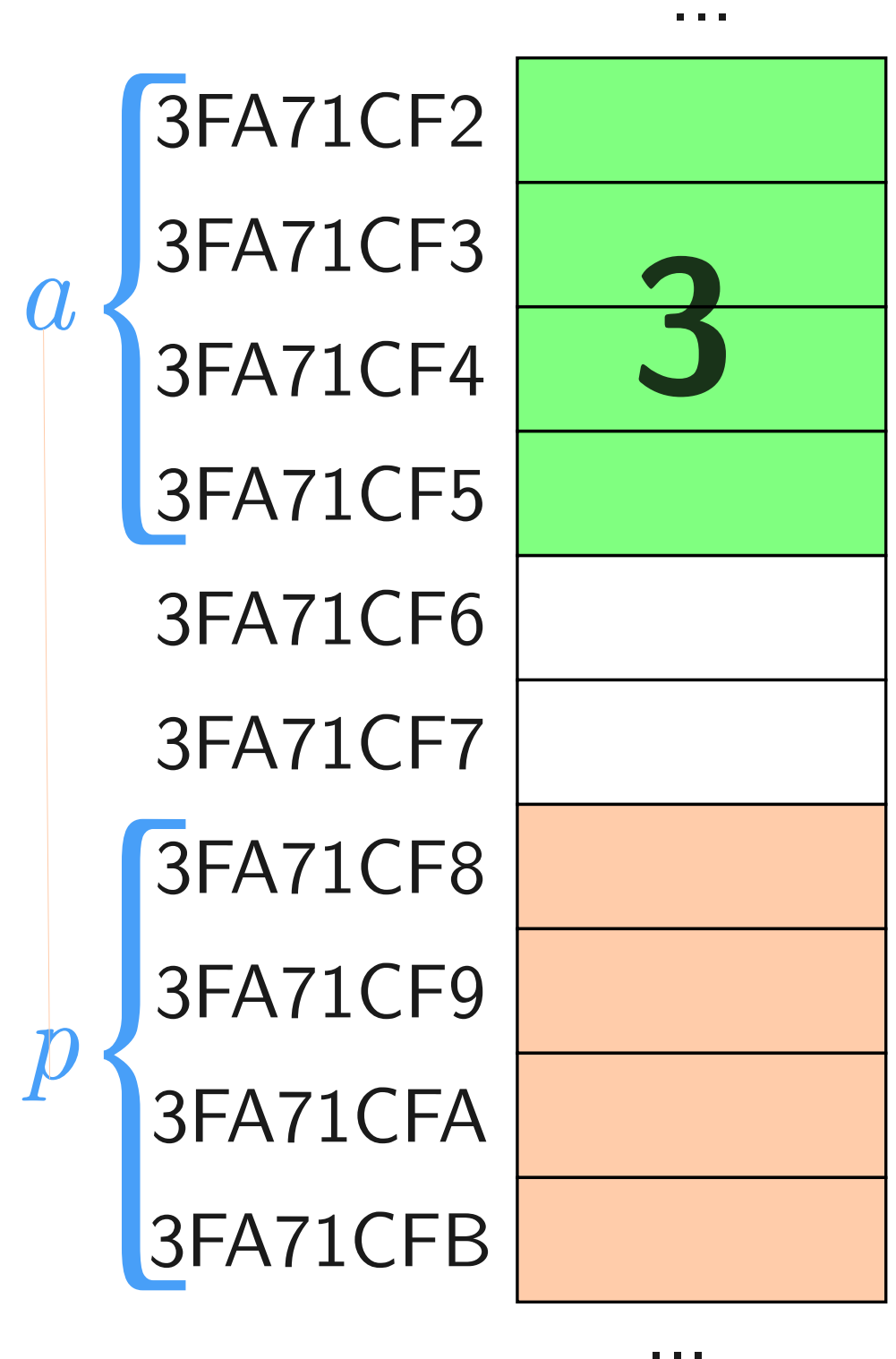
```
int main () {  
    int a, *p;  
  
    a = 3;  
    p = &a;  
    return 0;  
}
```



Atribuição Endereço a Ponteiro (2)

```
int main () {  
    int a, *p;
```

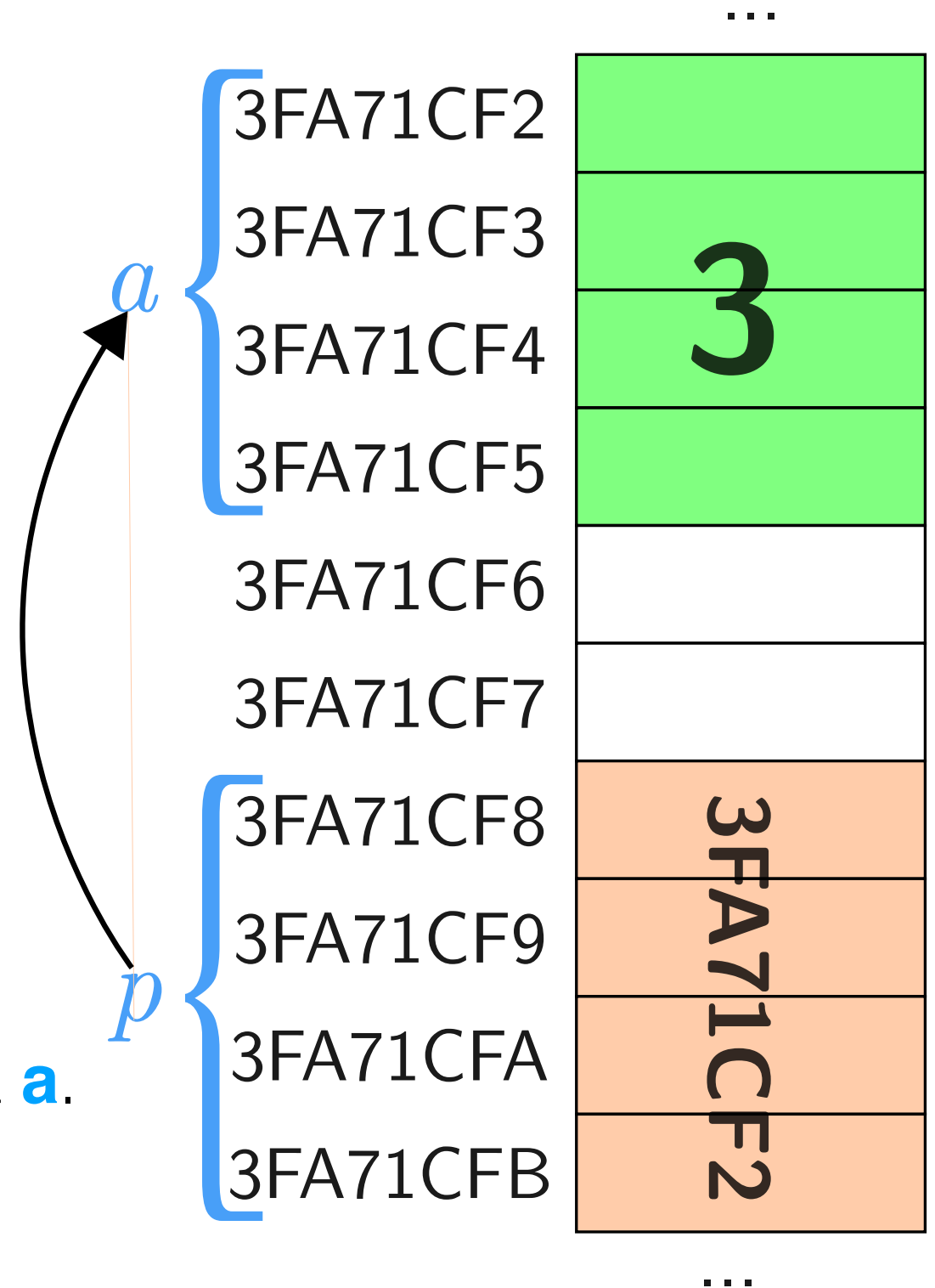
```
    a = 3;  
    p = &a;  
    return 0;  
}
```



Atribuição Endereço a Ponteiro (3)

```
int main () {  
    int a, *p;  
  
    a = 3;  
    p = &a;  
    return 0;  
}
```

Dizemos que o ponteiro **p** aponta para **a**.



Operador de Indireção (*)

- O **operador unário *** é conhecido como operador de indireção, e retorna o valor de um objeto no qual o ponteiro aponta
- * e & são inversos
 - Cancelam um ao outro

Conteúdo Ponteiro

Diferente do que ocorre com atribuição de variáveis simples ..

```
int main () {  
    int a, *p;
```

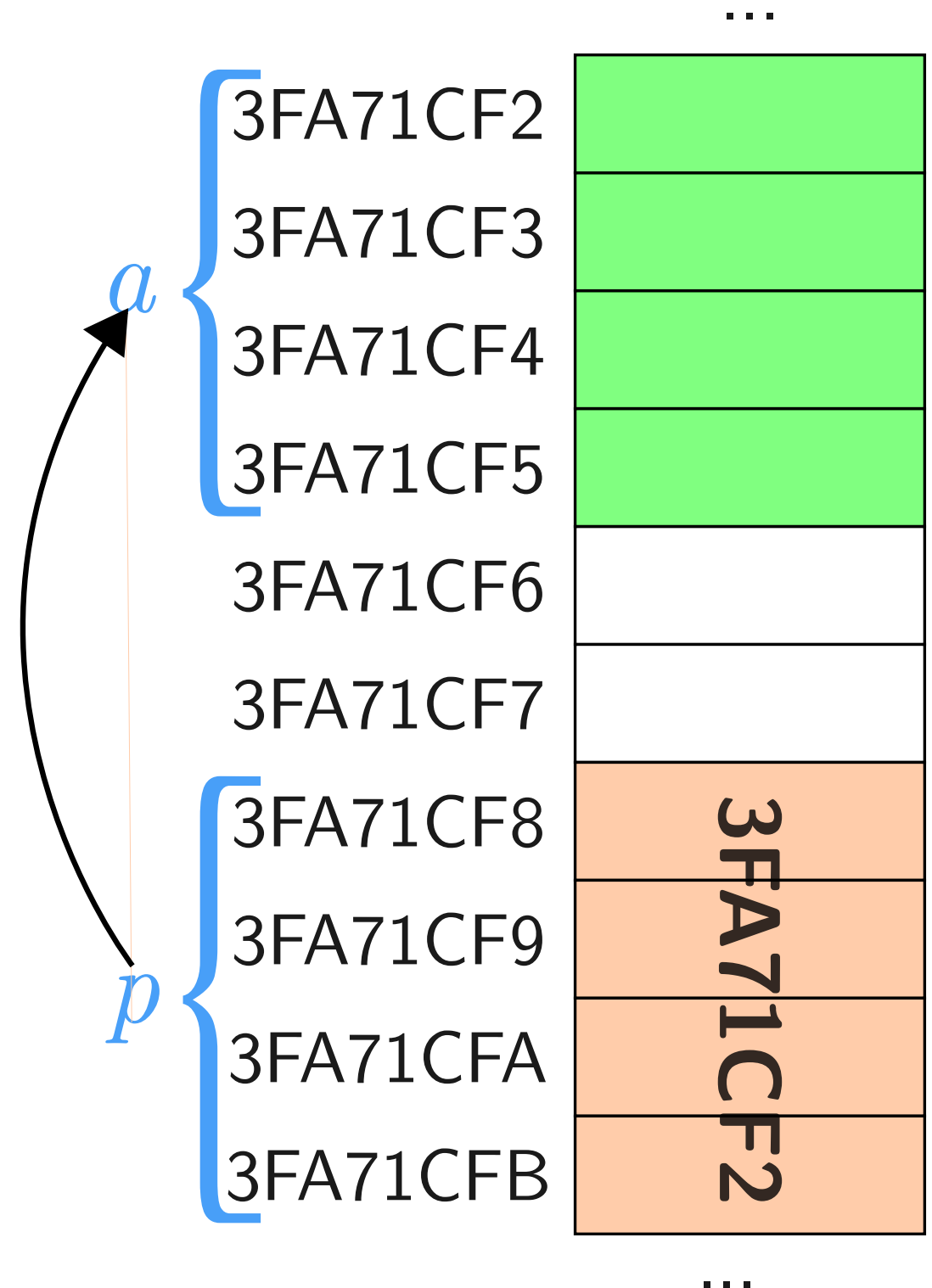
```
    p = &a;
```

```
    a = 5;
```

```
    *p = 3;
```

```
    return 0;
```

```
}
```



Conteúdo Ponteiro (2)

Diferente do que ocorre com atribuição de variáveis simples ..

```
int main () {  
    int a, *p;
```

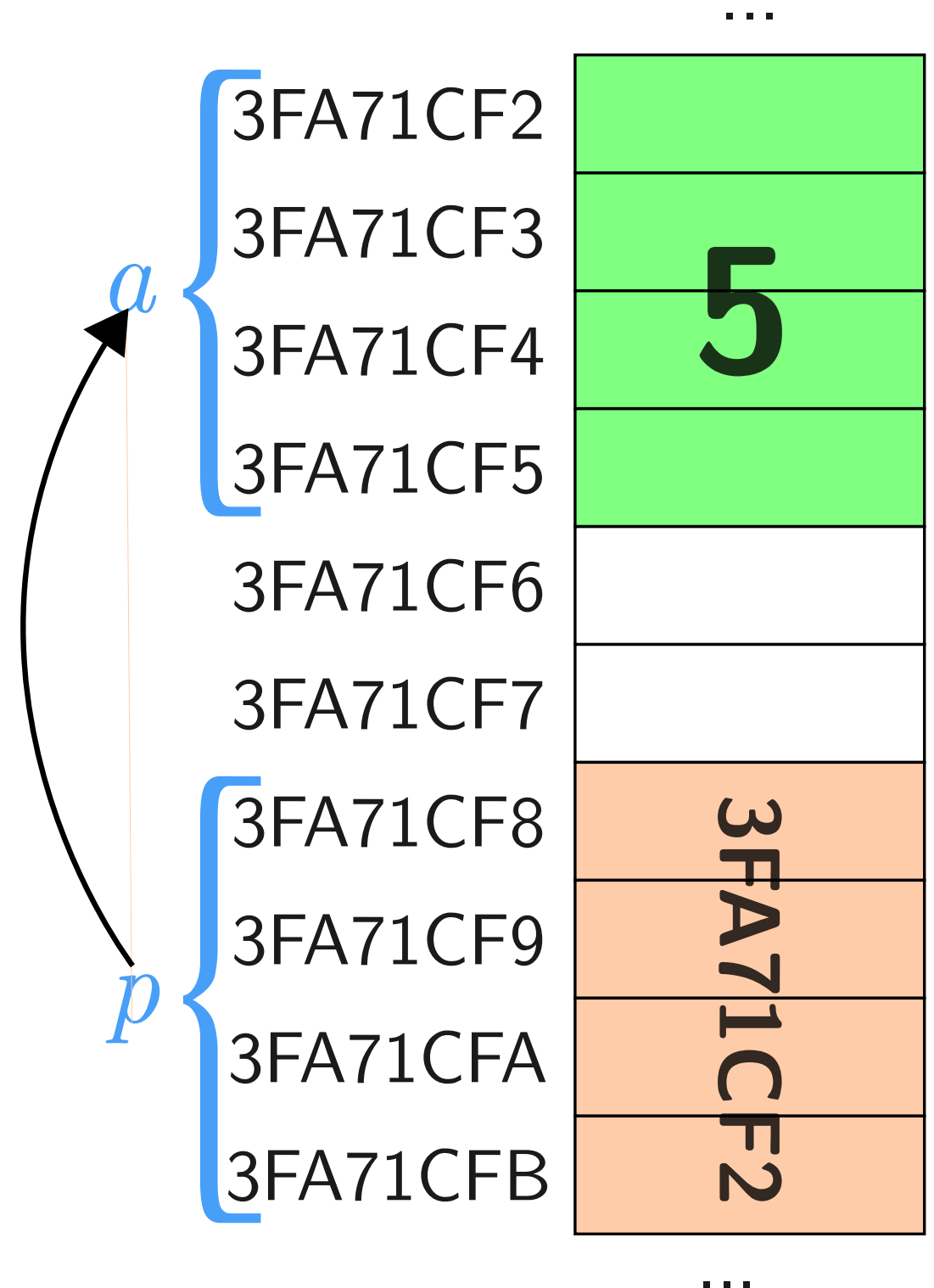
```
    p = &a;
```

```
    a = 5;
```

```
    *p = 3;
```

```
    return 0;
```

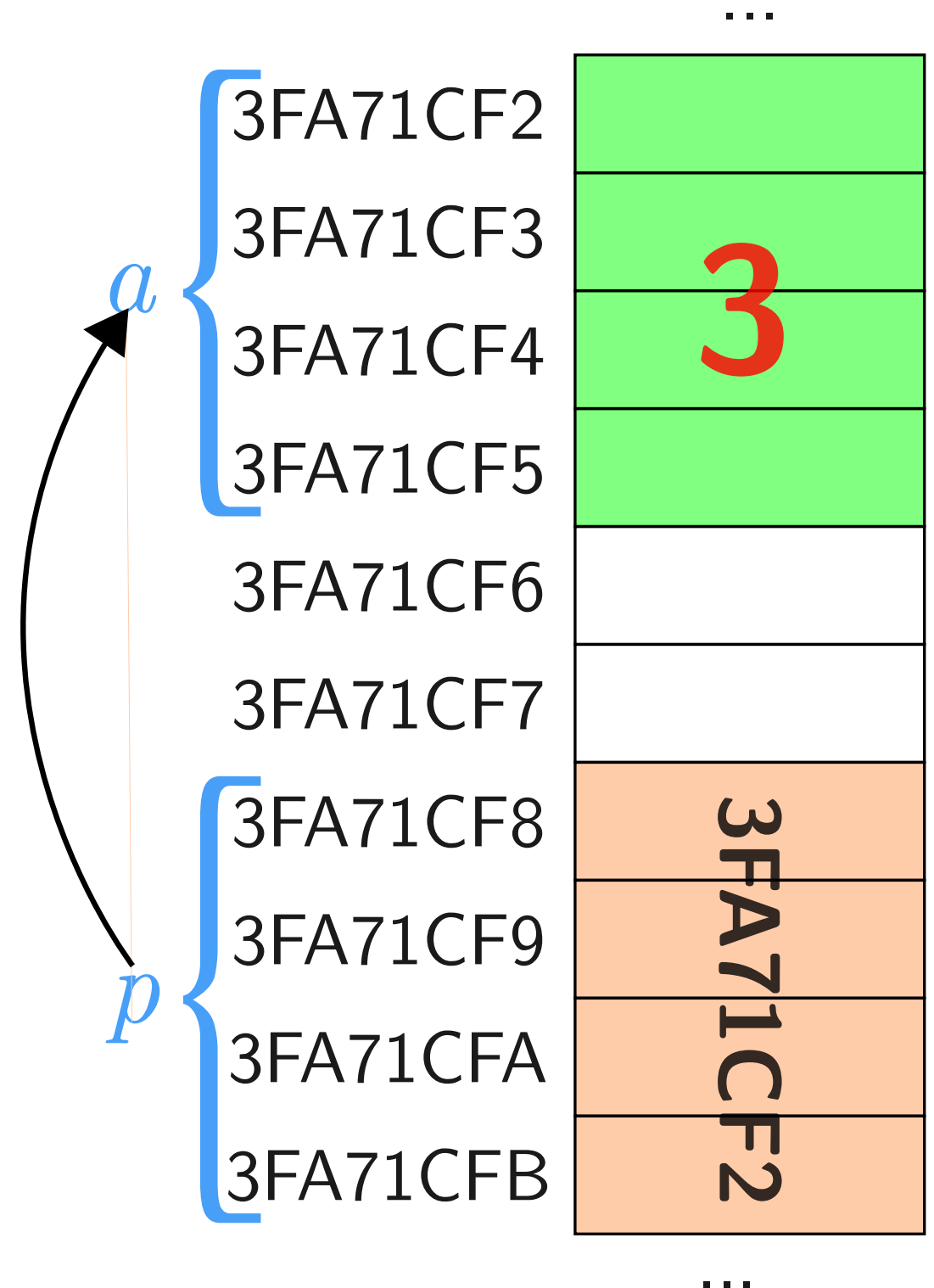
```
}
```



Conteúdo Ponteiro (3)

Diferente do que ocorre com atribuição de variáveis simples ..

```
int main () {  
    int a, *p;  
  
    p = &a;  
    a = 5;  
    *p = 3;  
    return 0;  
}
```



Erro Comum de Programação 2

Referenciar um ponteiro que não foi inicializado adequadamente ou que não foi atribuído para apontar para um local específico na memória é um erro. Isto pode causar um erro fatal em tempo de execução, ou pode acidentalmente modificar importantes dados e permitir que o programa execute até o fim com resultados incorretos.

```
#include <stdio.h>

int main() {
    int *p;

    *p = 3;
}
```

Erro Comum de Programação 3

Utilizar o operador de indireção (*) em uma variável não ponteiro é um erro sintático.

Erro Comum de Programação 4

Desreferenciar (aplicar o operador de indireção `*`) em um ponteiro que possui o valor `NULL` (0) gera um erro fatal em tempo de execução.

```
#include <stdio.h>

int main() {
    int *p = NULL;

    *p = 42; // Isto irá causar um erro de segmentação
}
```

```
zsh: segmentation fault
```


Operadores & e *

```
1. #include <stdio.h>
2.
3. int main ()
4. {
5.     int a, *aPtr;
6.
7.     a = 7;
8.     aPtr = &a;
9.
10.    printf("O endereco de a eh %p\n"
11.           "\nO valor de aPtr eh %p", &a, aPtr);
12.
13.    printf("\n\nO valor de a eh %d"
14.           "\nO valor de *aPtr eh %d", a, *aPtr);
15.
16.    printf("\n\nMostrando que * e & sao complementos"
17.           "\n&*aPtr = %p"
18.           "\n*&aPtr = %p\n", &*aPtr, *&aPtr);
19.
20.    return 0;
21. }
22.
```

Se aPtr aponta para a,
então &a e aPtr tem o
mesmo valor.

a e *aPtr tem o
mesmo valor

&*aPtr e *&aPtr tem o mesmo valor

Alocação de Memória

Alocação Estática

```
char c;  
int i;  
int v[10];
```

- Na **alocação estática** de memória o espaço para armazenar dados é reservado no início da execução do programa, e esse espaço permanece fixo durante toda a sua vida útil

Alocação Dinâmica

- Na **alocação dinâmica** de memória é um processo onde um programa reserva espaço na memória durante sua execução
- Isso é possível usando dos comandos **malloc** (para alocar a memória) e **free** (para liberar a memória)
 - Alocamos a nossa própria memória para o ponteiro
 - Estes comandos presentes na biblioteca **stdlib**

Exemplo: Alocação Dinâmica

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int i, *v, n;
6.
7.     printf("Entre com a quantidade de inteiros a serem lidos:\n");
8.     scanf("%d", &n);
9.
10.    // alogue memoria dinamicamente
11.    v = (int *) malloc(sizeof(int) * n);
12.
13.    // leia n inteiros
14.    printf("Entre com %d inteiros:\n", n);
15.    for(i = 0; i < n; i++)
16.        scanf("%d", &v[i]);
17.
18.    // imprima na ordem inversa da leitura
```

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int main() {
5.     int i, *v, n;
6.
7.     printf("Entre com a quantidade de inteiros a serem lidos:\n");
8.     scanf("%d", &n);
9.
10.    // aloque memoria dinamicamente
11.    v = (int *) malloc(sizeof(int) * n);
12.
13.    // leia n inteiros
14.    printf("Entre com %d inteiros:\n", n);
15.    for(i = 0; i < n; i++)
16.        scanf("%d", &v[i]);
17.
18.    // imprima na ordem inversa da leitura
19.    printf("Valores impressos na ordem inversa:\n");
20.    for(i = n-1; i >= 0; i--)
21.        printf("%d ", v[i]);
22.
23.    free(v);
24.    return 0;
25.}
```

Casting

v espera um valor do tipo (int *) malloc retorna uma variável do tipo (void *)

↓
v = (int *) malloc(sizeof(int) * n);
↑ É necessário fazer um casting

- Ponteiros do mesmo tipo podem ser atribuídos uns aos outros
 - Se não for do mesmo tipo, um operador **cast** deve ser utilizado
 - Exceção: ponteiro para **void** (tipo **void ***)
 - Nenhum **cast** necessário para converter um ponteiro para **void pointer**

(type_name) expression

Passagem de Parâmetros

Passagem de Parâmetros

Há três maneiras em C++ de passar argumentos para uma função:

1. Passagem por valor
2. Passagem por referência com argumentos de referência (somente em C++)
3. Passagem por referência com argumentos de ponteiro

1 - Passagem por Valor

- É a forma mais comum de passagem de parâmetros
- Uma cópia do valor do argumento é feita e passada para a função chamada
- Alterações na cópia não afetam valor variável original

Passagem por Referência

- Há situações que desejamos criar uma função que retorne mais de um valor
 - As funções que vimos até agora são capazes somente de retornar um único valor

2 - Passagem por Referência c/ Argumentos de Referência (Específico C++)

- Um **parâmetro de referência** é um *alias* para seu argumento correspondente em uma chamada de função
- Adicione um 'e comercial' (&) depois do tipo do parâmetro no cabeçalho da função
- Na chamada de função, simplesmente mencione a variável por nome para passá-la por referência

2 - Passagem por Referência c/ Argumentos de Referência (Específico C++): Exemplo

```
1. #include <stdio.h>
2.
3. int squareByValue(int n);
4. void squareByReference(int &n);
5.
6. int main() {
7.     int x = 2, z = 4;
8.
9.     // demonstra squareByValue
10.    printf("x = %d antes de squareByValue\n", x);
11.    printf("Valor retornado por squareByValue: %d\n", squareByValue(x));
12.    printf("x = %d depois squareByValue\n\n", x);
13.
14.    // demonstra squareByReference
15.    printf("z = %d antes de squareByReference\n", z);
16.    squareByReference(z);
17.    printf("z = %d depois squareByReference\n", z);
18.
19.    return 0;
20.}
21.
```

```

1. #include <stdio.h>
2.
3. int squareByValue(int n);
4. void squareByReference(int &n);
5.
6. int main() {
7.     int x = 2, z = 4;
8.
9.     // demonstra squareByValue
10.    printf("x = %d antes de squareByValue\n", x);
11.    printf("Valor retornado por squareByValue: %d\n", squareByValue(x));
12.    printf("x = %d depois squareByValue\n\n", x);
13.
14.    // demonstra squareByReference
15.    printf("z = %d antes de squareByReference\n", z);
16.    squareByReference(z);
17.    printf("z = %d depois squareByReference\n", z);
18.
19.    return 0;
20.}
21.
22. int squareByValue(int n) {
23.     return n = n * n;
24.}
25.
26. void squareByReference(int &nRef) {
27.     nRef = nRef * nRef;
28.}

```

x = 2 antes de squareByValue
Valor retornado por squareByValue: 4
x = 2 depois squareByValue

z = 4 antes de squareByReference
z = 16 depois squareByReference

Passagem por Referência com Argumentos de Ponteiro

- Em C, os programadores podem utilizar ponteiros e o operador de indireção (*) para realizar a passagem por referência
 - C não tem referências

Outline

fig07_07.c

```
1  /* Fig. 7.7: fig07_07.c
2     Cube a variable using call by reference with pointers */
3
4  #include <stdio.h>
5
6  void cubeByReference( int *nPtr ); /* prototype */
7
8  int main( void )
9  {
10     int number = 5; /* initialize number */
11
12     printf( "The original value of number is %d", number );
13
14     /* pass address of number to cubeByReference */
15     cubeByReference( &number );
16
17     printf( "\nThe new value of number is %d\n", number );
18
19     return 0; /* indicates successful termination */
20 } /* end main */
21
22
23 /* calculate cube of *nPtr; modifies variable number */
24 void cubeByReference( int *nPtr )
25 {
26     *nPtr = *nPtr * *nPtr * *nPtr; /* cube *nPtr */
27 } /* end function cubeByReference */
```

Protótipo da função recebe como argumento um ponteiro.

Função **cubeByReference** é passado um endereço, que pode ser o valor de uma variável ponteiro.

Neste programa, ***nPtr** é **number**, assim esta expressão modifica o valor de **number**.

The original value of number is 5
The new value of number is 125

Referências

- DEITEL, C Como Programar 6ª Edição. Pearson;
- FEOFILOFF, P. Algoritmos em Linguagem C, 1. ed. Rio de Janeiro: Elsevier, 2008;
- PIVA, D.J. et al. Algoritmos e programação de computadores. Rio de Janeiro: Elsevier, 2012;
- WIRTH, Niklaus. Algoritmos e estrutura de dados. Rio de Janeiro, RJ: LTC, 2009. 255p;